

CyberSentinel - Documentation.md

Integration Guide for Developers | Team Code Wizards | SIH 2026

CyberSentinel - Integration Guide for Developers

Connect any app - SaaS, e-commerce, chat, social, edtech, fintech - to CyberSentinel in ~15 minutes. No security expertise needed. Your app's end-users will notice nothing.

Who is this for?

You are a developer building or maintaining an app for your organization. You want CyberSentinel to watch your app for hacking, phishing, bot abuse, and account takeover.

You do not need to change your UI, database, or user flow. You only forward copies of security-relevant events from your backend to Sentinel.

1. How it works (in 30 seconds)

```
Your Backend (any stack: Node / Python / PHP / Java)
  login fails, password reset, link clicked, bulk API calls, file uploads
      |
      | POST /api/ingest (your backend -> Sentinel backend)
      v
CyberSentinel
  Score 0-100 -> Correlate 2-3 signals -> Detection -> Auto-Incident
      |
      v
Your Security Dashboard (Monitor / Graph / Detection / Incidents / Report)
```

- 1 weak signal = noise. Ignored.
- 2-3 related signals for the same user = attack. Alert + case opened.

Example: 5 failed logins + OTP failed + login from new country = Unauthorized Access, Risk 80. Or clicked suspicious link + submitted credentials + new-country login = Phishing -> Account Takeover, Risk 85.

2. Step 1 - Create your organization account

- 1. Open your Sentinel frontend (e.g. <http://localhost:5173>).
- 2. Go to Register.
- 3. Fill: email, password (8+ chars), your name, Organization Name (e.g. Acme Pvt Ltd).

What happens: Sentinel creates a private workspace for your company. All your data is isolated by organization. Your role is OWNER.

Login afterwards at Login with the same email/password.

3. Step 2 - Get your connection token

Option A: Permanent API Key (Recommended for server-to-server)

API keys are long-lived credentials (like Cloudinary's `cs_live_...`) designed for backend ingestion. They don't expire unless you set an expiry date.

- 1. Create an API key (requires OWNER or ADMIN role):

```
# First, login to get a JWT token
curl -X POST http://localhost:3001/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email": "dev@yourcompany.com", "password": "your-password"}'

# Then create an API key (save the returned key - it's shown only once!)
curl -X POST http://localhost:3001/api/auth/api-keys \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer <YOUR_JWT_TOKEN>" \
  -d '{"name": "production-ingest", "expiresInDays": 365}'
```

- 2. Copy the key from the response (format: `cs_live_...`).
- 3. In your app's `.env` file, add:

```
SENTINEL_URL=http://localhost:3001
SENTINEL_TOKEN=cs_live_...your-key
```

Option B: JWT Token (for interactive use)

- 1. Login via API:

```
curl -X POST http://localhost:3001/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email": "dev@yourcompany.com", "password": "your-password"}'
```

- 2. Copy the token from the response.
- 3. In your app's `.env` file, add:

```
SENTINEL_URL=http://localhost:3001
SENTINEL_TOKEN=eyJhbGciOi...your-token
```

Note: JWT tokens expire in 1 hour - re-login to get a new one if ingest returns 401 Invalid or expired token. API keys are recommended for production ingestion.

Send it on every request as:

```
Authorization: Bearer <SENTINEL_TOKEN>
```

3b. Managing API Keys

Once you have an API key, you can manage it via the API (requires OWNER or ADMIN role):

List all API keys:

```
curl -X GET http://localhost:3001/api/auth/api-keys \
  -H "Authorization: Bearer <YOUR_JWT_TOKEN>"
```

Get details of a specific API key:

```
curl -X GET http://localhost:3001/api/auth/api-keys/<KEY_ID> \
  -H "Authorization: Bearer <YOUR_JWT_TOKEN>"
```

Revoke an API key:

```
curl -X DELETE http://localhost:3001/api/auth/api-keys/<KEY_ID> \
-H "Authorization: Bearer <YOUR_JWT_TOKEN>"
```

Security note: The full API key is only returned once at creation time. Store it securely (e.g., in your secrets manager / .env). If lost, revoke and create a new one.

4. Step 3 - Add one helper to your backend

Create one file in your codebase. Node.js example (same idea in Python/PHP/Java/Go):

File: utils/sentinel.js (or sentinel.py, Sentinel.php - any name)

```
const SENTINEL_URL = process.env.SENTINEL_URL;
const SENTINEL_TOKEN = process.env.SENTINEL_TOKEN;

export async function reportToSentinel(signals) {
  if (!SENTINEL_URL || !SENTINEL_TOKEN) return;
  try {
    // fire-and-forget: don't block your user response
    fetch(`${SENTINEL_URL}/api/ingest`, {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        "Authorization": `Bearer ${SENTINEL_TOKEN}`
      },
      body: JSON.stringify({ signals })
    }).catch(() => {});
  } catch {}
}

export const now = () => new Date().toISOString();
```

Python equivalent:

```
import os, requests
def report_to_sentinel(signals):
    url = os.getenv("SENTINEL_URL")
    token = os.getenv("SENTINEL_TOKEN")
    if not url or not token: return
    try:
        requests.post(f"{url}/api/ingest",
            headers={"Authorization": f"Bearer {token}"},
            json={"signals": signals}, timeout=3)
    except: pass
```

You do not need to change your server setup. Just import this helper in your existing controllers.

5. Step 4 - Send events where they already happen

Call reportToSentinel() inside your existing logic. Pick the 3 patterns that match your app type:

A. Auth events - for EVERY app (login, signup, OTP, password reset)

In your login / verify-OTP / reset-password controller:

```
import { reportToSentinel, now } from "../utils/sentinel.js";

// FAILED login
reportToSentinel([
  {
    signalType: "BRUTE_FORCE_BURST",
    category: "AUTH",
    severity: "HIGH",
    message: Failed login for ${email} - attempt ${count},
    userIdentity: email, // must be same across related events
    sourceIp: req.ip,
    eventTimestamp: now(),
    rawData: { attempts: count }
  }
]);

// SUCCESS but suspicious (off-hours or new location/device)
if (isOffHours || isNewCountry) {
  reportToSentinel([
    { signalType: "OFF_HOURS_LOGIN", category: "AUTH", severity: "MEDIUM", message: Off-hours login ${email},
      userIdentity: email, sourceIp: req.ip, eventTimestamp: now(), rawData: {} },
    { signalType: "IMPOSSIBLE_TRAVEL", category: "AUTH", severity: "HIGH", message: ${email} Delhi then Lagos
      in 12 min, userIdentity: email, sourceIp: req.ip, eventTimestamp: now(), rawData: {} }
  ]);
}
```

Applies to: SaaS login, e-commerce checkout login, edtech student login, fintech wallet login - all the same.

B. User-content events - if your app has posts, messages, comments, reviews, uploads

In your create-post / send-message / upload controller:

```
if (containsLink(text) || isSpamBurst) {
  reportToSentinel([
    {
      signalType: "PHISH_CLICK",
      category: "WEB",
      severity: "MEDIUM",
      message: Suspicious link submitted by ${req.user.username},
      userIdentity: req.user.email || req.user.username,
      sourceIp: req.ip,
      domain: extractDomain(text), // e.g. "xn--paypa1.com"
      eventTimestamp: now(),
      rawData: { contentPreview: text.slice(0, 500) }
    }
  ]);
}
```

Examples: chat app DM with phishing link, social app spam post, e-commerce fake review with external URL, LMS assignment with malicious file link.

C. Privilege / sensitive-action events - if your app has roles, admin, payments, private files

On role change, refund, bulk export, private-file access:

```
reportToSentinel([
  {
    signalType: "PRIV_ESCALATION",
    category: "AUTH",
    severity: "HIGH",
    message: ${user.email} granted admin,
    userIdentity: user.email,
    sourceIp: req.ip,
```

```
eventTimestamp: now(),
rawData: {}
}]);
```

Examples: normal user -> admin, seller -> large refund, employee -> bulk customer-data download.

Golden rule (all apps): send 2+ related signals for the same userIdentity in one call when you can. That is what triggers a Detection. Singles alone are treated as noise.

Test with curl (works for any app):

```
curl -X POST http://localhost:3001/api/ingest \
-H "Authorization: Bearer $SENTINEL_TOKEN" \
-H "Content-Type: application/json" \
-d '{"signals":[
    {"signalType":"PHISH_CLICK","category":"EMAIL","severity":"MEDIUM","message":"Clicked
xn--paypa1.com","userIdentity":"adi@corp.com","eventTimestamp":"2026-09-26T10:00:00Z","rawData":{}},
    {"signalType":"CREDENTIAL_FORM_POST","category":"WEB","severity":"HIGH","message":"Password submitted on fake
page","userIdentity":"adi@corp.com","eventTimestamp":"2026-09-26T10:02:00Z","rawData":{}},
    {"signalType":"IMPOSSIBLE_TRAVEL","category":"AUTH","severity":"HIGH","message":"IN then NG
12min","userIdentity":"adi@corp.com","sourceIp":"103.21.244.10","eventTimestamp":"2026-09-26T10:14:00Z","rawData":{}},
    {}
]}'
# -> { processed: 3, detectionsTriggered: 1, incidentsCreated: 1 }
```

6. Event catalog - map YOUR app events to Sentinel

You map your app event -> Sentinel signalType. Use these exact uppercase names. Pick only rows relevant to your app:

Your app event (any domain)	Send as signalType	category	severity
5+ wrong passwords / OTPs in 1 min	BRUTE_FORCE_BURST	AUTH	HIGH
Login at 2-5am / weekend	OFF_HOURS_LOGIN	AUTH	MEDIUM
OTP / MFA failed 3x	MFA_FAILURE	AUTH	MEDIUM
Login from 2 countries in minutes	IMPOSSIBLE_TRAVEL	AUTH	HIGH
Normal user becomes admin / seller gets refund rights	PRIV_ESCALATION	AUTH	HIGH
Bulk private-data access (orders, student records, wallets)	SENSITIVE_SHARE_ACCESS	ENDPOINT	MEDIUM
User clicked / submitted suspicious link	PHISH_CLICK	EMAIL or WEB	MEDIUM
User submitted password / card on external page	CREDENTIAL_FORM_POST	WEB	HIGH
Account fires 100 API calls / messages in 1 min (bot)	BEACONING	NETWORK	MEDIUM
Request to unknown / newly-registered domain	SUSPICIOUS_DNS	NETWORK	MEDIUM
First-time connection to new region / ASN	NEW_ASN_CONN	NETWORK	LOW
Unknown script / binary / macro executed	RARE_PROCESS	ENDPOINT	MEDIUM
Mass file rename / encrypt (file-hosting, LMS, DMS apps)	MASS_FILE_RENAME	ENDPOINT	HIGH
Backup / shadow-copy deleted	SHADOW_COPY_DELETE	ENDPOINT	HIGH

How Sentinel groups them:

- PHISHING = PHISH_CLICK + CREDENTIAL_FORM_POST + IMPOSSIBLE_TRAVEL + MFA_FAILURE
- UNAUTH_ACCESS = BRUTE_FORCE_BURST + OFF_HOURS_LOGIN + PRIV_ESCALATION + SENSITIVE_SHARE_ACCESS
- MALWARE = RARE_PROCESS + BEACONING + SUSPICIOUS_DNS + NEW_ASN_CONN
- RANSOMWARE = MASS_FILE_RENAME + SHADOW_COPY_DELETE + HIGH_ENTROPY_WRITE + OUTBOUND_TOR

Score >=55 creates Detection, >=70 auto-opens Incident, >=85 is CRITICAL.

Which rows do I need?

- SaaS / edtech / internal tool -> auth rows + privilege rows.
- E-commerce / fintech -> auth rows + sensitive-access + suspicious-domain rows.
- Chat / social -> auth rows + link/bot rows.
- File-hosting / DMS -> all endpoint / ransomware rows.

7. Payload reference

POST /api/ingest - body:

```
{
  "signals": [
    {
      "signalType": "BRUTE_FORCE_BURST",
      "category": "AUTH",
      "severity": "HIGH",
      "message": "human-readable description (required)",
      "sourceIp": "103.21.244.10",
      "userIdentity": "user@yourapp.com",
      "hostname": "web-01",
      "domain": "evil.com",
      "deviceId": "optional-device-uuid",
      "rawData": { "any": "extra JSON" },
      "eventTimestamp": "2026-09-30T10:00:00.000Z"
    }
  ]
}
```

Rules:

- signals: array, 1-500 per request. Required.
- signalType: string, uppercased automatically. Required.
- category: one of NETWORK | AUTH | ENDPOINT | EMAIL | WEB. Defaults to NETWORK.
- severity: one of INFO | LOW | MEDIUM | HIGH | CRITICAL. Defaults to INFO.
- message: required, shown in dashboard feed.
- userIdentity, hostname: most important for correlation. Use stable IDs (user email / user-id + server/host). Same user + same host in one batch = grouped.
- eventTimestamp: ISO datetime. Use event time, not server time, if delayed.
- Response: { processed, detectionsTriggered, incidentsCreated }.

Limits: global 200 req / 15 min, auth 30 / 15 min. Batch events (e.g. every 5 sec) instead of one request per event.

8. Verify it works

- 1. Send the test curl from §5.
- 2. Open Sentinel -> Monitor - you should see your signals live.
- 3. Open Detection - you should see e.g. Probable Phishing -> Account Takeover, Risk ~85 with reasoning + weights.
- 4. Open Incidents - auto-created OPEN incident (score >=70). Click it -> assign -> change status -> add remediation note.

- 5. Open Graph (window=24h) - you should see user -> ip/domain nodes.
- 6. Open Report - counts by type/severity, top risky users.

If Monitor is empty: check token (401 = expired, re-login), check SENTINEL_URL port (backend defaults 3001, Vite proxy expects 5000 - set PORT to match), check browser network tab.

9. Daily workflow (after connection)

Your security team does this, not your app's end-users:

- 1. Home - totals: signals, detections in 24h, open incidents, devices.
- 2. Monitor - live feed. Filter by severity / category / search / minRisk.
- 3. Graph - attack path. Click a node to see risk. Use 1h / 24h / 7d.
- 4. Detection - open one -> read reasoning + matched signals + confidence -> Promote to incident if real.
- 5. Incidents - assign teammate, move OPEN -> INVESTIGATING -> CONTAINED -> RESOLVED (or FALSE_POSITIVE / DISMISSED), log actions like BLOCKED_IP, RESET_PASSWORD, DISABLED_ACCOUNT.
- 6. Report - pick date range -> charts by type/severity + risky-users table -> export JSON for management.

Your app users never see any of this. They keep using your app normally.

10. Best practices (any stack)

- Fire-and-forget. Never await Sentinel in your user request path. Send in background so Sentinel downtime never breaks your app.
- Send security events only. Logins, password changes, links, spam bursts, admin changes, payments/refunds. Do not send every page view or chat message - you will hit rate limits and noise.
- Stable IDs. Always send same userIdentity (user-id or email, not display name that changes) and hostname (server/pod name).
- Batch. Collect 10-50 events for 5 seconds, send once. Better correlation + fewer requests.
- Keep rawData small. First 500 chars, counts, user-agent. Never send passwords, tokens, card numbers, or full PII bodies.
- Handle 401. If ingest returns 401, refresh your login token and retry once.
- One service account. Create one Sentinel user like bot@yourcompany.com just for ingest, don't use a human's token.

11. Troubleshooting

Symptom	Cause	Fix
401 Unauthorized	Invalid or expired token	JWT expired (1h) or missing Bearer Re-login POST /api/auth/login, update SENTINEL_TOKEN
401 Invalid API key	API key revoked, expired, or malformed	Create new API key via POST /api/auth/api-keys, check cs_live_ prefix
429 Too many requests	Hit 200/15min limiter	Batch more, reduce polling
400 validation error	Missing signalType/message/eventTimestamp or bad category/severity enum	Check \$7 enums, ISO date
Monitor/Graph empty	No signals in window, wrong org token, PORT mismatch	Re-send test curl, check window=7d, confirm same org login

| Detection not created | Only 1 weak signal sent, or different userIdentity | Send 2+ matching types for same user in one batch (see §6) |
| CORS error in browser | FRONTEND_URL mismatch | Set backend FRONTEND_URL to match frontend origin |

12. FAQ

Do my app users need a Sentinel account? No. Only your team logs in. App users just appear as userIdentity strings.

Do I need to install an agent / SDK? No. Just the HTTPS push from §4. No daemon, no dependency.

Which languages work? Any backend that can send HTTPS POST with JSON - Node, Python, PHP, Java, Go, Ruby, .NET.

Does this slow my app? No if you use fire-and-forget (§10). Scoring runs on Sentinel side.

Is my data mixed with others? No. Every query filters by organization. You only see your org.

Can I use this in production? For hackathon/demo yes. Before production add: role check on key-creation routes, and 30-min history correlation (currently only current batch is correlated). Permanent API keys are now implemented.

What does it cost? Nothing extra. Postgres + Node only. No external AI API needed.

13. Quick checklist (any developer)

- ☐ Registered org + logged in
- ☐ Copied token -> your app .env as SENTINEL_TOKEN (+ SENTINEL_URL)
- ☐ Added 1 helper (reportToSentinel / report_to_sentinel)
- ☐ Added calls in existing code: auth events + content/link events (if any) + privilege events (if any)
- ☐ Sent test curl -> saw Detection + Incident
- ☐ Team knows Monitor -> Detection -> Incidents -> Report flow

Done. Your app - whatever it does - is now watched.